55247e59-7add-40a1-9810-afd0bafb0511

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'Kane610/axis' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/axis/axis (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-06-26T21:58:33.178+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Add VAPIX temperature control MVP handler and parsing
</pr_title>

<pr_description>
Summary
- add a new temperature control model module for `/axis-cgi/temperaturecontrol.cgi?action=statusall`
- add a new temperature control handler and register it in Vapix API discovery initialization
- parse `statusall` text output into typed items for sensor/heater/fan entries
- normalize device status values with support for `Running[XX]` intensity extraction
- add dedicated tests for temperature parsing and handler update

```
behavior
- update Vapix integration tests and request/response contract
tests

## Scope
This is MVP read-only support for `statusall`.
- Included: discovery-wired handler + text parsing + typed model
+ tests
- Not included: control actions (`start/stop`), per-device query
endpoints, param.cgi trigger updates, param.cgi fallback
initialization

## Validation
- `uv run pytest tests/test_temperature_control.py tests/
test_vapix.py tests/test_request_response_contracts.py`
- `uv run ruff check axis tests`
- `uv run mypy axis`

## Notes
- `uv.lock` was intentionally left out of this PR to keep scope
focused on temperature API implementation.
</pr_description>

## Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>









 float | None = None
    name: str | None = None

@dataclass(frozen=True)
class TemperatureActuator(TemperatureDevice):  # Heater and Fan
are structurally identical
    intensity: int | None = None
    status: TemperatureDeviceStatus =
TemperatureDeviceStatus.UNKNOWN
    status_raw: str | None = None
    time_until_stop: int | None = None
```

```
@dataclass(frozen=True)
class TemperatureHeater(TemperatureActuator): ...
@dataclass(frozen=True)
class TemperatureFan(TemperatureActuator): ...
```

Key observations:
- `TemperatureHeater` and `TemperatureFan` are **field-identical today**. Making them separate leaf classes is justified because the VAPIX API treats them as distinct device categories (different ID namespaces, different control actions in the future). A shared `TemperatureActuator` intermediate avoids the duplication.
- `ApiHandler[TemperatureDevice]` works cleanly: the internal `_items: dict[str, TemperatureDevice]` stores all three types via polymorphism. The typed accessors (`sensors`, `heaters`, etc.) then return the narrowed types.
- `GetStatusAllResponse` returns `dict[str, TemperatureDevice]`. The dispatch at decode time (`Sensor.*` → `TemperatureSensor(...)`, etc.) replaces the current `_device_type_from_id` heuristic with factory dispatch.
- The `_missing_` enum and all helper functions (`_parse_status`, `_to_float`, `_to_int`) are unaffected.

The one genuine concern: type checkers cannot exhaustively narrow a `dict[str, TemperatureDevice]` at the call site — callers still need `isinstance()` for ad-hoc access via `handler["Sensor.S0"]`. The typed accessors (`get_sensor(id) -> TemperatureSensor | None`, `sensors -> dict[str, TemperatureSensor]`) solve this for the primary use path; raw dict access is a legitimate escape hatch.

Verdict: Do it. The refactor is moderate in scope (~50 lines net change), eliminates the discriminator field, and produces strictly better types on all the public accessor methods already present in the handler.

Grumpy UX Architect

The problem is the false optionals. Right now, if I call `handler.get_sensor("Sensor.S0")` and get back a `TemperatureControlStatus`, I see `status: TemperatureDeviceStatus` and `intensity: int | None` in my IDE. Those fields do not apply to a sensor. I don't know if `intensity is None` means "not running" or "sensors don't have

intensity" or "device didn't report it." **Ambiguous `None` is the worst kind of lie in a typed API.**

The existing typed accessors — `get_sensor()`, `sensors`, `get_heater()`, etc. — are already the right idea. They just return the wrong type (`TemperatureControlStatus` instead of a narrowed type), so the type system gives up exactly where it should be most helpful.

With the subclass split, `get_sensor()` returns `TemperatureSensor | None`. The irrelevant fields are gone. The IDE autocomplete shows only `celsius`, `fahrenheit`, `name`. A developer cannot accidentally read `sensor.intensity`. That is what a typed library should do.

The `TemperatureActuator` intermediate is worth the small extra complexity because it lets `running_heaters` and `running_fans` both return `dict[str, TemperatureActuator]`, allowing code that operates on "any running actuator" to work without a union type.

Verdict: The split is the right call for a public library API. The `device_type` enum should be removed entirely when the classes are the type. The only thing that should stay is the `TemperatureDeviceStatus` enum — that one is legitimately shared state.

Bottom line

Both perspectives agree: split is worth doing. The hierarchy to reach for is `TemperatureDevice` → `TemperatureSensor` + `TemperatureActuator` → `TemperatureHeater` + `TemperatureFan`. The handler stays as a single `ApiHandler[TemperatureDevice]`; the typed accessors get their narrowed return types; `TemperatureControlStatus` and `TemperatureDeviceType` are retired. This is a contained refactor — the tests change in assertions (`isinstance(sensor, TemperatureSensor)` instead of `isinstance(sensor, TemperatureControlStatus)`) but the fixture and mock structure stays the same. Happy to implement if you want to proceed.
</comment_old>

```
<comment_new>
<comment_id>4813707931</comment_id>
<author>@Kane610</author>
@copilot Give me a descriptive suggestion on how this would
look and how it alignes with how its done in general to this
```

```
library
</comment_new>
```

</pr_comments>

</comments>

The last **12** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. Treat <comment_old> comments as context only and do not make changes solely to satisfy them.
* Only act on <comment_new> comments that explicitly mention you (@copilot). To understand the context of comments that explicitly mention you, you may use other comments that do not explicitly mention you. Do not act on comments that don't explicitly mention you or reply to them, even if they are marked as <comment_new>.
* If the PR author has pushed back on or rejected a suggestion in a thread, do not act on that thread.
* For each comment you decide to act on, determine:
    - If it is a request for changes, a question on the code, or a suggestion.
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

<system_reminder>

</system_reminder>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.

<prompt>

@copilot Give me a descriptive suggestion on how this would look and how it alignes with how its done in general to this library

<prev_assistant>

I've replied to the comment with a detailed code sketch. Here's a summary of what was covered:

What the refactor looks like:
- `TemperatureDevice(ApiItem)` — abstract base (only `id`, no device-specific fields)
- `TemperatureSensor(TemperatureDevice)` — `celsius`, `fahrenheit`, `name`
- `TemperatureActuator(TemperatureDevice)` — shared `status`, `intensity`, `status_raw`, `time_until_stop` for both heaters and fans
- `TemperatureHeater(TemperatureActuator)` and `TemperatureFan(TemperatureActuator)` — distinct leaf classes with no extra fields today, justified by VAPIX API category separation
- `TemperatureDeviceType` enum retired — class identity is the type

How it aligns with library conventions:
- `ApiHandler[TemperatureDevice]` uses the exact same generic pattern as `LightHandler[LightInformation]`, `ApiHandler[PirSensorConfiguration]`, etc.
- Typed accessors use `isinstance` narrowing instead of `device_type == X` comparisons — which is what mypy needs to infer the correct return type
- The factory dispatch in `GetStatusAllResponse.decode()` is cleaner and eliminates the `_device_type_from_id` heuristic
- Helper functions (`_parse_status`, `_parse_statusall_entries`, `_to_float`, `_to_int`) are fully preserved
- Net change is ~50 lines with no change to test fixture shape

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'Kane610/axis' repository.

<repository_context>

I've cloned the repository in the directory /home/runner/work/axis/axis (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-06-26T22:02:24.863+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Add VAPIX temperature control MVP handler and parsing
</pr_title>

<pr_description>
Summary
- add a new temperature control model module for `/axis-cgi/temperaturecontrol.cgi?action=statusall`
- add a new temperature control handler and register it in Vapix API discovery initialization
- parse `statusall` text output into typed items for sensor/heater/fan entries
- normalize device status values with support for `Running[XX]` intensity extraction
- add dedicated tests for temperature parsing and handler update behavior
- update Vapix integration tests and request/response contract tests

Scope
This is MVP read-only support for `statusall`.
- Included: discovery-wired handler + text parsing + typed model + tests
- Not included: control actions (`start/stop`), per-device query endpoints, param.cgi trigger updates, param.cgi fallback initialization

Validation
- `uv run pytest tests/test_temperature_control.py tests/test_vapix.py tests/test_request_response_contracts.py`
- `uv run ruff check axis tests`
- `uv run mypy axis`

```
## Notes
- `uv.lock` was intentionally left out of this PR to keep scope
focused on temperature API implementation.
</pr_description>

## Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>









actoring.guru/smells/type-codes">type code smell</a>: a field
whose sole purpose is to tell you which fields are actually
meaningful. In OO terms, that's what subclasses are for.

**The right shape with subclasses:**

```python
@dataclass(frozen=True)
class TemperatureDevice(ApiItem):          # abstract base
    ...

@dataclass(frozen=True)
class TemperatureSensor(TemperatureDevice):
    celsius: float | None = None
    fahrenheit: float | None = None
    name: str | None = None

@dataclass(frozen=True)
class TemperatureActuator(TemperatureDevice):  # Heater and Fan
are structurally identical
    intensity: int | None = None
    status: TemperatureDeviceStatus =
TemperatureDeviceStatus.UNKNOWN
    status_raw: str | None = None
    time_until_stop: int | None = None

@dataclass(frozen=True)
class TemperatureHeater(TemperatureActuator): ...
@dataclass(frozen=True)
class TemperatureFan(TemperatureActuator): ...
```
```

```

Key observations:
- `TemperatureHeater` and `TemperatureFan` are **field-identical today**. Making them separate leaf classes is justified because the VAPIX API treats them as distinct device categories (different ID namespaces, different control actions in the future). A shared `TemperatureActuator` intermediate avoids the duplication.
- `ApiHandler[TemperatureDevice]` works cleanly: the internal `_items: dict[str, TemperatureDevice]` stores all three types via polymorphism. The typed accessors (`sensors`, `heaters`, etc.) then return the narrowed types.
- `GetStatusAllResponse` returns `dict[str, TemperatureDevice]`. The dispatch at decode time (`Sensor.*` → `TemperatureSensor(...)`, etc.) replaces the current `_device_type_from_id` heuristic with factory dispatch.
- The `_missing_` enum and all helper functions (`_parse_status`, `_to_float`, `_to_int`) are unaffected.

**The one genuine concern:** type checkers cannot exhaustively narrow a `dict[str, TemperatureDevice]` at the call site — callers still need `isinstance()` for ad-hoc access via `handler["Sensor.S0"]`. The typed accessors (`get_sensor(id) -> TemperatureSensor | None`, `sensors -> dict[str, TemperatureSensor]`) solve this for the primary use path; raw dict access is a legitimate escape hatch.

**Verdict:** Do it. The refactor is moderate in scope (~50 lines net change), eliminates the discriminator field, and produces strictly better types on all the public accessor methods already present in the handler.

---

## Grumpy UX Architect

The problem is the false optionals. Right now, if I call `handler.get_sensor("Sensor.S0")` and get back a `TemperatureControlStatus`, I see `status: TemperatureDeviceStatus` and `intensity: int | None` in my IDE. Those fields do not apply to a sensor. I don't know if `intensity is None` means "not running" or "sensors don't have intensity" or "device didn't report it." **Ambiguous `None` is the worst kind of lie in a typed API.**

The existing typed accessors — `get_sensor()`, `sensors`,
```

`get_heater()`, etc. — are already the right idea. They just return the wrong type (`TemperatureControlStatus` instead of a narrowed type), so the type system gives up exactly where it should be most helpful.

With the subclass split, `get_sensor()` returns `TemperatureSensor | None`. The irrelevant fields are gone. The IDE autocomplete shows only `celsius`, `fahrenheit`, `name`. A developer cannot accidentally read `sensor.intensity`. That is what a typed library should do.

The `TemperatureActuator` intermediate is worth the small extra complexity because it lets `running_heaters` and `running_fans` both return `dict[str, TemperatureActuator]`, allowing code that operates on "any running actuator" to work without a union type.

Verdict: The split is the right call for a public library API. The `device_type` enum should be removed entirely when the classes are the type. The only thing that should stay is the `TemperatureDeviceStatus` enum — that one is legitimately shared state.

Bottom line

Both perspectives agree: split is worth doing. The hierarchy to reach for is `TemperatureDevice` → `TemperatureSensor` + `TemperatureActuator` → `TemperatureHeater` + `TemperatureFan`. The handler stays as a single `ApiHandler[TemperatureDevice]`; the typed accessors get their narrowed return types; `TemperatureControlStatus` and `TemperatureDeviceType` are retired. This is a contained refactor — the tests change in assertions (`isinstance(sensor, TemperatureSensor)` instead of `isinstance(sensor, TemperatureControlStatus)`) but the fixture and mock structure stays the same. Happy to implement if you want to proceed.
</comment_old>

<comment_old>
<author>@Kane610</author>
@copilot Give me a descriptive suggestion on how this would look and how it alignes with how its done in general to this library
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot Give me a descriptive suggestion on how this would look and how it alignes with how its done in general to this library

Here is a concrete sketch of what the refactor would look like alongside the library's established patterns.

How it maps to existing conventions

The library already uses `ApiItem` as the typed unit of a handler's item store. Each model file defines **one concrete `ApiItem` subclass** per logical entity (e.g. `LightInformation`, `PirSensorConfiguration`, `PortItem`). The `ApiHandler[T]` generic constrains the internal `_items` dict to that type, and typed accessors on the handler class narrow where needed.

The temperature model currently fits that shape except for one issue: a single `TemperatureControlStatus` class carries fields from all three device kinds, gated behind a `device_type` discriminator. Splitting into subclasses removes the discriminator and keeps each class's field set honest, which is exactly what Python dataclass inheritance is for.

Suggested model layout (`axis/models/temperature_control.py`)

```python
# ── shared infrastructure ──────────────────────────────────────────

class TemperatureDeviceStatus(enum.StrEnum):
    FAILURE = "failure"
    RUNNING  = "running"
    STOPPED  = "stopped"
    UNKNOWN  = "unknown"

    @classmethod
    def _missing_(cls, value: object) ->
TemperatureDeviceStatus:
        return cls.UNKNOWN

# TemperatureDeviceType is retired — class identity carries that
```

```
information.

# ── base class
─────────────────────────────────────────────────────────────────


@dataclass(frozen=True)
class TemperatureDevice(ApiItem):
    """Abstract base for any temperature-managed device."""

    @classmethod
    def decode(cls, data: dict[str, str]) -> Self:          #
stays abstract in practice
        raise NotImplementedError

# ── leaf classes
─────────────────────────────────────────────────────────────


@dataclass(frozen=True)
class TemperatureSensor(TemperatureDevice):
    """A temperature-reporting sensor (Sensor.Sx entries)."""

    celsius: float | None = None
    fahrenheit: float | None = None
    name: str | None = None

    @classmethod
    def decode(cls, data: dict[str, str]) -> Self:
        return cls(
            id=data["id"],
            celsius=_to_float(data.get("Celsius")),
            fahrenheit=_to_float(data.get("Fahrenheit")),
            name=data.get("Name"),
        )


@dataclass(frozen=True)
class TemperatureActuator(TemperatureDevice):
    """Shared fields for Heater and Fan devices."""

    intensity: int | None = None
    status: TemperatureDeviceStatus =
TemperatureDeviceStatus.UNKNOWN
    status_raw: str | None = None
    time_until_stop: int | None = None

    @classmethod
    def decode(cls, data: dict[str, str]) -> Self:
```

```
        status_raw = data.get("Status")
        status, intensity = _parse_status(status_raw)
        return cls(
            id=data["id"],
            intensity=intensity,
            status=status,
            status_raw=status_raw,
            time_until_stop=_to_int(data.get("TimeUntilStop")),
        )


@dataclass(frozen=True)
class TemperatureHeater(TemperatureActuator):
    """A controllable heater (Heater.Hx entries)."""


@dataclass(frozen=True)
class TemperatureFan(TemperatureActuator):
    """A controllable fan (Fan.Fx entries)."""
```

`TemperatureHeater` and `TemperatureFan` are structurally identical today but are distinct VAPIX device categories with separate ID namespaces and future control actions. Leaf classes without extra fields are fine — the library uses the same pattern in the application models (`LoiteringGuard`, `MotionGuard`, etc. all share an `Application` base with minimal per-subclass changes).

The response decoder becomes a factory dispatch:

```python
_DEVICE_FACTORY: dict[str, type[TemperatureDevice]] = {
    "Sensor": TemperatureSensor,
    "Heater": TemperatureHeater,
    "Fan":    TemperatureFan,
}

@classmethod
def decode(cls, bytes_data: bytes) -> GetStatusAllResponse:
    payload = bytes_data.decode("utf-8")
    parsed  = _parse_statusall_entries(payload)
    data: dict[str, TemperatureDevice] = {}
    for prefix, factory in _DEVICE_FACTORY.items():
        for item_id, fields in parsed[prefix.lower()].items():
            data[item_id] = factory.decode(fields)
    return cls(data=data)
```

```

---

## Suggested handler layout (`axis/interfaces/
temperature_control.py`)

```python
class TemperatureControlHandler(ApiHandler[TemperatureDevice]):
    """Temperature control status for Axis devices."""

    api_id            = ApiId.TEMPERATURE_CONTROL
    default_api_version = API_VERSION
    handler_groups    = (HandlerGroup.API_DISCOVERY,)

    async def _api_request(self) -> dict[str,
TemperatureDevice]:
        return await self.get_status_all()

    async def get_status_all(self) -> dict[str,
TemperatureDevice]:
        response = await
self.vapix.api_request(GetStatusAllRequest())
        return response.data

    # ── narrowed typed accessors
──────────────────────────────────────────

    @property
    def sensors(self) -> dict[str, TemperatureSensor]:
        return {k: v for k, v in self.items() if isinstance(v,
TemperatureSensor)}

    @property
    def heaters(self) -> dict[str, TemperatureHeater]:
        return {k: v for k, v in self.items() if isinstance(v,
TemperatureHeater)}

    @property
    def fans(self) -> dict[str, TemperatureFan]:
        return {k: v for k, v in self.items() if isinstance(v,
TemperatureFan)}

    def get_sensor(self, sensor_id: str) -> TemperatureSensor |
None:
        item = self.get(sensor_id)
        return item if isinstance(item, TemperatureSensor) else
```
```

````
None

    def get_heater(self, heater_id: str) -> TemperatureHeater |
None:
        item = self.get(heater_id)
        return item if isinstance(item, TemperatureHeater) else
None

    def get_fan(self, fan_id: str) -> TemperatureFan | None:
        item = self.get(fan_id)
        return item if isinstance(item, TemperatureFan) else
None

    @property
    def running_heaters(self) -> dict[str, TemperatureHeater]:
        return {k: v for k, v in self.heaters.items()
                if v.status == TemperatureDeviceStatus.RUNNING}

    @property
    def running_fans(self) -> dict[str, TemperatureFan]:
        return {k: v for k, v in self.fans.items()
                if v.status == TemperatureDeviceStatus.RUNNING}
```

The `isinstance` guards are the library-idiomatic way to narrow
`ApiItem` subclasses; they replace the `device_type ==
TemperatureDeviceType.X` comparisons and are what mypy needs to
infer the correct return type on the accessors.

---

## What changes, what stays

| | Current | After refactor |
|---|---|---|
| Public item classes | `TemperatureControlStatus` |
`TemperatureSensor`, `TemperatureHeater`, `TemperatureFan` |
| Discriminator enum | `TemperatureDeviceType` | **removed** |
| Handler generic | `ApiHandler[TemperatureControlStatus]` |
`ApiHandler[TemperatureDevice]` |
| Accessor return types | `dict[str, TemperatureControlStatus]`
| `dict[str, TemperatureSensor]` etc. |
| `isinstance` checks in handler | `item.device_type == …` |
`isinstance(item, TemperatureSensor)` |
| `_parse_statusall_entries` | unchanged | unchanged |
| Helper functions | unchanged | unchanged |
| Test fixture/mock shape | unchanged | assertion types change
````
````

```
to subclasses |

Happy to proceed with the implementation if this direction looks
good.
</comment_old>

  <comment_new>
  <comment_id>4813726300</comment_id>
  <author>@Kane610</author>
  @copilot ok, execute this and add it to a new commit and
  push it
  </comment_new>

</pr_comments>

</comments>

----
The last **12** git commits in this branch are the changes you
have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the
<comments> before making any changes. Treat <comment_old> comments
as context only and do not make changes solely to satisfy them.
* Only act on <comment_new> comments that explicitly mention you
(@copilot). To understand the context of comments that explicitly
mention you, you may use other comments that do not explicitly
mention you. Do not act on comments that don't explicitly mention
you or reply to them, even if they are marked as <comment_new>.
* If the PR author has pushed back on or rejected a suggestion in a
thread, do not act on that thread.
* For each comment you decide to act on, determine:
    - If it is a request for changes, a question on the code, or a
suggestion.
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can
stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and
ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>
```

<system_reminder>

</system_reminder>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.

<prompt>

@copilot ok, execute this and add it to a new commit and push it